PYTHON ACTIVITY-BASED LEARNING

Case Study Solutions

Student Name	Vishal
Roll Number	26001028
Subject	Python Programming

Selected Case Studies 1–10

Each case includes a problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, sample expected result, and conclusion. Values and rules not supplied in the question sheet are illustrative assumptions; replace them if your faculty has given different rules.

1. University Scholarship Allocation

Problem Statement

Evaluate applicants using aggregate marks, subject-wise minimum scores, attendance, and family income; rank eligible students and resolve ties.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Assume three subjects; each mark must be at least 40, average marks at least 75, attendance at least 75%, and annual family income at most Rs. 250,000. Rank by average marks, then attendance.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START
INPUT required values
VALIDATE input values
CALCULATE totals and apply conditions
DISPLAY result
STOP
```

Python Source Code

```
applicants = [
    ("Asha", [88, 81, 90], 92, 180000),
    ("Ravi", [78, 80, 76], 85, 220000),
    ("Meena", [70, 72, 68], 90, 200000),
    ("Kiran", [91, 89, 94], 70, 150000)
]

eligible = []
for name, marks, attendance, income in applicants:
    average = sum(marks) / len(marks)
    if (min(marks) >= 40 and average >= 75
            and attendance >= 75 and income <= 250000):
        eligible.append((name, average, attendance))

eligible.sort(key=lambda x: (x[1], x[2]), reverse=True)
for rank, student in enumerate(eligible, 1):
    print(rank, student)
```

Sample Test Case / Expected Result

Expected: Asha and Ravi qualify. Asha ranks first because her average marks are higher.

Conclusion

The program applies eligibility conditions and ranks successful applicants using a defined tie-breaking rule.

2. Progressive Electricity Billing

Problem Statement

Calculate household electricity bills using consumption slabs, fixed charges, surcharges, rebates, and month-to-month comparison.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

First 100 units: Rs. 2/unit; next 100: Rs. 3/unit; above 200: Rs. 5/unit. Fixed charge Rs. 50, surcharge 5% of energy charge, and Rs. 20 rebate for consumption up to 100 units. A rise above 30% is unusual.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START
INPUT required values
VALIDATE input values
CALCULATE totals and apply conditions
DISPLAY result
STOP
```

Python Source Code

```
def energy_charge(units):
    charge = min(units, 100) * 2
    if units > 100:
        charge += min(units - 100, 100) * 3
    if units > 200:
        charge += (units - 200) * 5
    return charge

previous_bill = None
for month, units in enumerate([90, 110, 260], 1):
    energy = energy_charge(units)
    bill = energy + 50 + energy * 0.05
    if units <= 100:
        bill -= 20
    print("Month", month, "bill Rs.", round(bill, 2))
    if previous_bill is not None and (bill - previous_bill) / previous_bill > 0.30:
        print("Unusual increase detected")
    previous_bill = bill
```

Sample Test Case / Expected Result

Expected: Month 1 bill is Rs. 219.00. A substantial increase in later months is flagged.

Conclusion

The program calculates slab-based bills and detects unusually large increases.

3. Hospital Billing and Patient Classification

Problem Statement

Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate amounts, and classify payment status.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Patients aged 60 or older receive 10% off consultation and treatment charges; diagnostic charges are not discounted. Negative charges or payments are invalid.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START
INPUT required values
VALIDATE input values
CALCULATE totals and apply conditions
DISPLAY result
STOP
```

Python Source Code

```
age = 65
consultation = 500
diagnostics = 1200
treatment = 4000
paid = 4000

if min(consultation, diagnostics, treatment, paid) < 0:
    print("Invalid billing data")
else:
    concession = 0.10 * (consultation + treatment) if age >= 60 else 0
    total = consultation + diagnostics + treatment - concession
    balance = total - paid
    status = "Paid" if balance <= 0 else "Pending"
    print("Final bill: Rs.", total)
    print("Balance: Rs.", max(0, balance))
    print("Status:", status)
```

Sample Test Case / Expected Result

Expected: Final bill Rs. 5250.0; balance Rs. 1250.0; status Pending.

Conclusion

The program validates billing values, applies an age concession, and classifies the payment status.

4. Employee Payroll and Deduction Analysis

Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and print a salary statement.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Overtime pay equals overtime hours multiplied by the hourly rate. Deduction is 0% on the first Rs. 10,000, 5% on the next Rs. 10,000, and 10% above Rs. 20,000.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START
INPUT required values
VALIDATE input values
CALCULATE totals and apply conditions
DISPLAY result
STOP
```

Python Source Code

```
basic = 20000
allowances = 5000
overtime_hours = 10
hourly_rate = 200
incentive = 1500

gross = basic + allowances + overtime_hours * hourly_rate + incentive
deduction = 0
if gross > 10000:
    deduction += min(gross - 10000, 10000) * 0.05
if gross > 20000:
    deduction += (gross - 20000) * 0.10

net_salary = gross - deduction
print("Gross salary: Rs.", gross)
print("Deduction: Rs.", deduction)
print("Net salary: Rs.", net_salary)
```

Sample Test Case / Expected Result

Expected: Gross salary Rs. 28,500; deduction Rs. 1,850; net salary Rs. 26,650.

Conclusion

The program calculates salary components and progressive deductions to obtain net salary.

5. Multi-Plan Telecommunications Cost Optimization

Problem Statement

Compare mobile plans based on data usage, validity, and budget; include excess-data charges and select the cheapest eligible plan.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Required validity is 28 days. Each plan is represented by its name, price, included GB, validity days, and extra-GB rate.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START
INPUT required values
VALIDATE input values
CALCULATE totals and apply conditions
DISPLAY result
STOP
```

Python Source Code

```
usage = 12
budget = 500
plans = [
    ("Basic", 249, 8, 28, 20),
    ("Smart", 399, 15, 56, 15),
    ("Max", 499, 25, 56, 10)
]

eligible = []
for name, price, included, days, extra_rate in plans:
    cost = price + max(0, usage - included) * extra_rate
    print(name, "cost Rs.", cost)
    if days >= 28 and cost <= budget:
        eligible.append((cost, name))

if eligible:
    print("Best plan:", min(eligible)[1])
else:
    print("No eligible plan")
```

Sample Test Case / Expected Result

Expected: Basic costs Rs. 329 and is the cheapest eligible plan for the supplied usage and budget.

Conclusion

The program compares plan costs after excess usage charges and chooses the lowest-cost eligible plan.

6. Railway Fare and Reservation Calculator

Problem Statement

Calculate passenger fares using distance slabs/classes, age concessions, reservation fees, and cancellation penalties.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Sleeper fare is Rs. 1/km and AC fare is Rs. 2.50/km; reservation fee is Rs. 40. Children under 12 receive 50% off fare, and passengers aged 60 or above receive 20% off fare. Cancellation penalty is 25% of discounted fare plus reservation fee.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START
INPUT required values
VALIDATE input values
CALCULATE totals and apply conditions
DISPLAY result
STOP
```

Python Source Code

```
passengers = [
    ("Anu", 10, 300, "Sleeper", False),
    ("Ramesh", 65, 300, "AC", True)
]
rates = {"Sleeper": 1.0, "AC": 2.5}

for name, age, distance, travel_class, cancelled in passengers:
    if distance <= 0 or age < 0 or travel_class not in rates:
        print(name, "Invalid details")
        continue
    fare = distance * rates[travel_class]
    if age < 12:
        fare *= 0.50
    elif age >= 60:
        fare *= 0.80
    total = fare + 40
    print(name, "total Rs.", total)
    if cancelled:
        penalty = 0.25 * fare + 40
        print("Penalty Rs.", penalty, "Refund Rs.", max(0, total - penalty))
```

Sample Test Case / Expected Result

Expected: Anu's total is Rs. 190. Ramesh's AC fare including reservation before cancellation is Rs. 640.

Conclusion

The program validates passenger details, applies age concessions, and calculates fare and cancellation amounts.

7. Retail Billing with Multiple Discount Rules

Problem Statement

Create an itemized retail invoice using product quantities, category-specific taxes, threshold discounts, and a maximum concession.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Tax rates: groceries 5%, clothing 12%, electronics 18%. A 10% discount applies when subtotal is at least Rs. 5,000, capped at Rs. 1,000. Tax is calculated after discount.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START
INPUT required values
VALIDATE input values
CALCULATE totals and apply conditions
DISPLAY result
STOP
```

Python Source Code

```
items = [
    ("Rice", 800, 2, "groceries"),
    ("Shirt", 1200, 2, "clothing"),
    ("Headphones", 2500, 1, "electronics")
]
tax_rates = {"groceries": 0.05, "clothing": 0.12, "electronics": 0.18}
subtotal = sum(price * qty for name, price, qty, category in items)
discount = min(subtotal * 0.10, 1000) if subtotal >= 5000 else 0
discount_share = discount / subtotal if subtotal else 0
tax_total = 0

for name, price, qty, category in items:
    discounted_line = price * qty * (1 - discount_share)
    tax = discounted_line * tax_rates[category]
    tax_total += tax
    print(name, "line Rs.", price * qty, "tax Rs.", round(tax, 2))

print("Subtotal:", subtotal, "Discount:", discount)
print("Tax:", round(tax_total, 2))
print("Final amount:", round(subtotal - discount + tax_total, 2))
```

Sample Test Case / Expected Result

Expected: Subtotal Rs. 6,800; discount Rs. 680; final amount is subtotal minus discount plus category taxes.

Conclusion

The program creates an itemized invoice with category-based taxes and a capped threshold discount.

8. Manufacturing Cost and Profitability Analysis

Problem Statement

Calculate material cost, wastage, labour, machine costs, sales revenue, profit, and whether the target profit is achieved.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Wastage is 5% of material cost. Target profit is 20% of total production cost.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START
INPUT required values
VALIDATE input values
CALCULATE totals and apply conditions
DISPLAY result
STOP
```

Python Source Code

```
quantity = 100
material_per_unit = 80
labour = 2500
machine = 1200
selling_price = 125

material = quantity * material_per_unit
wastage = material * 0.05
total_cost = material + wastage + labour + machine
revenue = quantity * selling_price
profit = revenue - total_cost
profit_percent = profit / total_cost * 100

print("Total cost:", total_cost)
print("Revenue:", revenue)
print("Profit:", profit)
print("Profit percentage:", round(profit_percent, 2))
print("Target met" if profit_percent >= 20 else "Target not met")
```

Sample Test Case / Expected Result

Expected from the stated values: total cost Rs. 12,100; revenue Rs. 12,500; profit Rs. 400; the 20% target is not met.

Conclusion

The program estimates production costs and checks profitability against a specified target.

9. Multi-Currency Travel Budget Analysis

Problem Statement

Convert travel expenses into INR using supplied exchange rates, include conversion fees, compare against a budget, and identify the largest expense category.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Exchange rates are expressed as INR per unit of foreign currency. A 2% conversion fee is applied to each converted expense.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START
INPUT required values
VALIDATE input values
CALCULATE totals and apply conditions
DISPLAY result
STOP
```

Python Source Code

```
expenses = [
    ("Hotel", 200, "USD"),
    ("Food", 80, "EUR"),
    ("Transport", 50, "USD")
]
rates = {"USD": 83, "EUR": 90}
budget = 25000
totals = []

for category, amount, currency in expenses:
    converted = amount * rates[currency]
    total = converted * 1.02
    totals.append((category, total))
    print(category, "INR incl. fee:", round(total, 2))

grand_total = sum(value for category, value in totals)
print("Total:", round(grand_total, 2))
print("Largest category:", max(totals, key=lambda x: x[1])[0])
print("Budget status:", "Within budget" if grand_total <= budget else "Over budget")
```

Sample Test Case / Expected Result

Expected: Total spending Rs. 28,509; largest category Hotel; budget exceeded.

Conclusion

The program converts expenses to INR, includes conversion fees, and checks the overall budget.

10. Campus Event Financial Planning

Problem Statement

Calculate event costs, participant contributions, sponsorship shortfalls, and the minimum registration count needed to break even.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Venue, equipment, and transport are fixed costs. Catering and certificates are per-participant costs. Registration fee is Rs. 500 and sponsorship is fixed.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START
INPUT required values
VALIDATE input values
CALCULATE totals and apply conditions
DISPLAY result
STOP
```

Python Source Code

```
venue = 12000
equipment = 5000
transport = 3000
catering = 250
certificate = 50
sponsorship = 8000
registration_fee = 500
participants = 100

fixed_cost = venue + equipment + transport
variable_per_person = catering + certificate
total_cost = fixed_cost + variable_per_person * participants
income = sponsorship + registration_fee * participants
shortfall = max(0, total_cost - income)
net_per_person = registration_fee - variable_per_person

needed = max(0, (fixed_cost - sponsorship + net_per_person - 1) // net_per_person)
print("Total event cost:", total_cost)
print("Total income:", income)
print("Shortfall:", shortfall)
print("Minimum registrations to break even:", needed)
```

Sample Test Case / Expected Result

Expected: Total cost Rs. 50,000; income Rs. 58,000; shortfall Rs. 0; minimum registrations to break even is 30.

Conclusion

The program estimates event finances and determines the registrations needed to cover costs.